

LLVM Infrastructure Cheatsheet

L0 Essence: LLVM 是一个基于强类型 SSA 同构 IR 构建的、将前端解析、中端 Pass 独立优化、后端 SelectionDAG / 机器 IR / 寄存器分配解耦的现代编译器基础设施与工具链。

[M1.1] 静态单赋值 (SSA) 设计哲学、Phi 节点与 Def-Use 链

LLVM IR 的灵魂是静态单赋值 (Static Single Assignment, SSA) 形式: `%%textbf1.%%textbfSSA` 规范: 每个虚拟寄存器只能在物理上被赋值一次。这极大地简化了中端的数据流分析与优化算法 (如常量折叠与公共子表达式消除)。`%%textbf2.%%textbfPhi` 节点: 为了表达控制流汇合点 (如 `if-else` 分支后) 的变量选择, 引入 `phi` 指令。它根据控制流“前驱基本块 (Predecessor)”的来源动态决定当前的变量值。`%%textbf3.%%textbfDef-Use & Use-Def` 链: 内存中每个 `Value` 类都维护着由其使用者组成的链表 (Def-Use 链), 而 `User` 类维护其引用的参数列表 (Use-Def 链)。这使得编译器可以在 $O(1)$ 的时间内完成死代码判定与常量传播。

[M1.2] LLVM IR 的三种同构表示 Module/Function

LLVM 保证其 IR 在编译的整个生命周期中具有完全同构的表达能力, 支持三种等价形态: `%%textbf1.%%textbf内存表示 (In-memory Class)`: 由 C++ 对象组成。最顶层是 `llvm::Module` (表示一个编译单元), 包含 `llvm::Function` (函数列表)、`llvm::GlobalVariable` (全局变量列表) 以及 `llvm::BasicBlock` (基本块)。基本块是线性指令序列的容器。`%%textbf2.%%textbf磁盘 Bitcode (.bc)`: 紧凑的二进制紧缩编码格式, 便于磁盘持久化与编译器快速读取加载。`%%textbf3.%%textbf文本汇编 (.ll)`: 人类可读的文本表示, 便于调试和 Pass 分析排查。

[M1.3] 强类型系统物理定义与安全验证

LLVM 拥有一个与目标平台无关的强类型系统, 在编译早期就规避了非法内存访问与类型隐式转换: `%%textbf1.%%textbf基本类型`: 包含任意宽度的整型 (如 `i1` 对应布尔, `i8` 对应字节, `i32/i64` 等)、浮点型 (`float/double`) 与 `void` 类型。`%%textbf2.%%textbf派生类型`: 包含数组 (`[N x Type]`)、结构体 (`Typel, Type2`)、指针 (`ptr`) 及面向 SIMD 向量化优化的向量类型 (`<v x Type>`)。`%%textbf3.%%textbf安全验证`: 任何类型转换必须显式使用转换指令 (如整型截断 `trunc`、无符号扩展 `zext`、指针转换 `bitcast` 等), 禁止任何隐式截断。

[M1.4] 基本块 CFG 约束与终结符 Terminator

基本块 (BasicBlock) 是 LLVM IR 流程控制的基本原子物理单位: `%%textbf1.%%textbf基本块结构`: 满足“单入口, 单出口”的控制流图 (CFG) 约束。每个基本块除了第一条指令外, 其余指令均不可被外部直接跳转跳转, 除最后一条指令外, 其他指令也无法跳转出去。`%%textbf2.%%textbf终结符指令 (Terminator)`: 必须是基本块的最后一条指令。用于控制流程流向, 包含无条件跳转 `br`、条件分支 `br i1`、函数返回 `ret`、跳转表 `switch`、或异常抛出 `resume/cleanupret`。`%%textbf3.%%textbfEntry 块约束`: 函数的第一个基本块 (Entry) 绝对不能拥有任何 CFG 前驱节点, 以保证入口的确定性。

[M1.5] alloca 栈变量、load/store 内存访问抽象

为了简化前端编译器的实现, LLVM 并不要求前端直接生成 SSA 格式的 IR: `%%textbf1.%%textbf内存分配抽象`: 前端可以将所有的局部变量平铺在栈上, 通过 `alloca` 指令分配内存。`%%textbf2.%%textbf访存接口`: 读写局部变量使用 `load` 和 `store` 访存指令。由于栈变量是在内存中, 不属于 SSA 虚拟寄存器, 因此可以多次被 `store` 写入, 绕开了 SSA 的“单点赋值”限制。`%%textbf3.%%textbf中端提升`: 这一步会产生大量冗余的内存读写, 需要依赖中端的 `Mem2Reg` Pass 将其重命名提升为寄存器 SSA 结构。

[M2.1] Clang AST 遍历与 IRBuilder 生成翻译

Clang 前端解析生成 AST 树, 随后通过 AST Visitor 访问模式生成 IR 模块: `%%textbf1.%%textbf类角色`: `clang::CodeGen::CodeGenModule` 维护全局生成上下文; `clang::CodeGen::CodeGenFunction` 维护函数内变量作用域、当前生成基本块与栈空间。`%%textbf2.%%textbfIRBuilder`: 提供 `llvm::IRBuilder<>` 工具类。在 AST 遍历 (如语句节点 `Stmt`、表达式节点 `Expr`) 时, 流式地创建 LLVM IR 指令, 并自动向当前插入点 (Insertion Point) 追加指令节点。`%%textbf3.%%textbf常量折叠`: IRBuilder 在创建指令时, 会对操作数进行即时常量分析, 若能进行折叠 (如 `1+2` 自动转为 `3`) 则直接返回常量对象, 避免在中端制造垃圾指令。

[M2.2] 控制流分支 br 条件降级与基本块拆分

高层语言复杂的流程控制结构 (如 `if-else`、`while` / `for` 循环) 需要被翻译为扁平的基本块跳转关系: `%%textbf1.%%textbfIf-Else 降级`: `%%textbf2.%%textbf拆分`为三个新基本块: `then` 基本块, `else` 基本块和 `merge` 汇合块。`%%textbf3.%%textbf在当前块根据条件表达式计算出 i1 结果`, 调用 `br i1 %cond, label %then, label %else` 进行条件跳转。`%%textbf4.%%textbf在 then 和 else 基本块的末尾, 强制插入 br label %merge` 无条件跳转汇合。`%%textbf2.%%textbf短路求值`: 逻辑与 `&&`、逻辑或 `||` 表达式会被流式拆解成多个级联的条件跳转基本块, 实现早期求值短路保护。

[M2.3] 调用约定 ABI、参数传递与 landingpad 异常注册

函数调用涉及到 ABI (Application Binary Interface) 的底座对齐与运行时异常捕获: `%%textbf1.%%textbfABI 降级`: Clang 需要根据目标平台的 Calling Convention (如 `x86_64-sysv`) 决定参数如何布局。超出物理寄存器宽度的结构体参数可能会被隐式转换为指针 (`byval` 属性), 或者拆解为多个整型进行参数传递。`%%textbf2.%%textbf异常捕获`: C++ 等高级语言的 `try-catch` 需要翻译为 LLVM 的异常处理指令: `%%textbf3.%%textbf调用可能抛出异常的函数不能使用普通的 call, 必须使用 invoke`。`%%textbf4.%%textbfinvoke 指定正常流跳转的基本块, 以及异常分支的汇合块 unwind`。`%%textbf5.%%textbf在 unwind 汇合块首行, 必须声明 landingpad 指令, 注册当前能捕获的类型元数据 (TypeInfo)`, 未匹配的异常会继续向外传播 `resume`。

[M2.4] 局部变量生存期 lifetime 与 Dwarf 调试信息

前端需要在 IR 中保留调试信息与高级生存期分析锚点, 以备后端优化和调试器使用: `%%textbf1.%%textbf生存期标志`: 编译器在作用域入口和出口处分别插入内置函数调用 `llvm.lifetime.start` 与 `llvm.lifetime.end`。中端优化 Pass (如 `StackColoring`) 据此对生命周期不重叠的栈变量进行合并, 大幅压缩物理栈帧大小。`%%textbf2.%%textbfDIBuilder (Debug Information Builder)`: 用于生成元数据, 它以 Dwarf 格式向 IR 节点挂载额外的说明。如局部变量元数据 `!DILocalVariable` 标明了源码行号、变量名, 并用 `llvm.dbg.declare` 或 `llvm.dbg.value` 追踪变量在 SSA 变换后的物理位置。

[M2.5] IR 验证器 verifyModule 静态完备性校验

在前端将 IR 交给中端优化链、或者中端 Pass 刚运行完毕后, 必须强制运行 IR 验证器: `%%textbf1.%%textbf静态验证`: `llvm::verifyModule()` 会遍历遍历 Module 内所有的函数与基本块, 执行语义检查。`%%textbf2.%%textbf验证维度`: `%%textbf3.%%textbfSSA 支配性约束 (Dominance)`: 任何 SSA 变量 `Use` 指令的物理位置必须被其定义 `Def` 所在的指令所“支配” (即执行路径必然先经过 Def 块)。`%%textbf4.%%textbf终结符合法性`: 每个基本块的最后一条指令必须且仅能是一条 Terminator 指令, 禁止中途出现跳转。`%%textbf5.%%textbf类型完备`: 校验操作数类型一致性 (如二进制加法 `add` 两侧的类型必须完全一致, 严禁 `i32` 与 `i64` 直接相加)。

L1 Logical Framework

`%%textbf1.%%textbfSSA 同构设计与多流表示`: LLVM 采用静态单赋值形式的 IR, 以同构的内存类、磁盘 bitcode、汇编文本三态流通, 确保优化分析无缝解耦。`%%textbf2.%%textbfPass 驱动中端与解耦提升`: 统一的 PassManager 调度分析与转换 Pass, 核心 `Mem2Reg` 基于支配边界将栈上 `alloca` 提升为 SSA 虚拟寄存器, 驱动后续死代码消除和循环优化。`%%textbf3.%%textbf合法选择与平铺机器指令`: 后端通过 SelectionDAG 将 IR 进行类型和操作合法化, TableGen 规则映射物理指令, 指令调度器将其重新平铺为线性 MachineIR 序列。`%%textbf4.%%textbf分配着色与物理文件发射`: 寄存器分配器根据活跃区间干涉图着色并处理溢出, PEI 构造真实物理栈帧, 最终在 MC 层以段布局发射 ELF/Mach-O 二进制文件。

L2 Data Flow Topology

`%%textbf1.%%textbfSource Code → Clang AST Visitor → llvm::IRBuilder → Raw LLVM IR (Alloca/Load/Store) → llvm::verifyModule check → PassManager (Mem2Reg) → SSA IR (Registers & Phi) → LICM/SimplifyCFG passes → Optimized IR → SelectionDAGISel construction → SelectionDAG Graph → Type/Operation Legalize → ISel Pattern Match (TableGen .td) → Target-Specific DAG → ListScheduler Scheduling → MachineInstr (MI) → Live Interval Calculation → RegAllocGreedy (Interference Graph Coloring) → Spill check? Yes → Insert spill load/store → PEI (Prologue/Epilogue) → MCStreamer emitter → MCInst Stream → ELF Object File / OrcJIT Execution.`

Trade-off Matrix: Optimizations & CodeGen

`%%textbf1.%%textbf局部变量表达形式`: 局部栈分配访存模型 (Alloca + Load/Store) → 纯静态单赋值寄存器模型 (Pure SSA Registers) [前端代码生成极为简单, 无需考虑复杂的控制流变量合并与作用域分析 → 但制造了海量内存冗余读写, 数据流完全断裂: 纯 SSA 在生成期就要通过复杂的算法推导分支合并并插入 Phi, 算法复杂 → 但有利于极致的中端分析。[LLVM 选择在前端使用 Alloca 规避前端复杂性, 在中端通过开山 Pass Mem2Reg 提升为纯 SSA 寄存器, 实现各阶段的最优平衡]] `%%textbf2.%%textbf指令选择流水线`: 传统的 SelectionDAG 机制 (DAG-based ISel) → 新一代的 GlobalSel 机制 (gMIR-based ISel) [SelectionDAG 将基本块转换为图, 类型和操作的合法化极其精准, 匹配度极高 → 但每次构造 DAG 会产生数值的内存开销与图遍历耗时, 且无法跨基本块全局优化: GlobalSel 在编译全程使用线性的通用 MachineIR (gMIR) 流通, 内存极小, 支持全局 Pass → 但早期目标架构 TableGen 规则支持度较低 [对于快速 JIT 与调试编译, 选用 GlobalSel: 对于高阶静态代码生成优化, 选用 SelectionDAG]] `%%textbf3.%%textbf寄存器分配策略`: 基于图着色的 Kempe 算法 (Graph Coloring RegAlloc) → 基于局部代价分析的贪婪算法 (Greedy RegAlloc) [图着色算法将分配映射为 NP-Complete 的图着色问题, 在变量较少时能给出理论最优分配 → 但在大型程序中, 由于其需要迭代重构干涉图, 编译速度较慢且易发生死锁: 贪婪分配器采用启发式区间拆分、溢出开销评估并结合局部扫描, 编译速度快且易于扩展 → 可能会在个别边界情况损失少量寄存器利用率 [LLVM 默认采用 Greedy 算法, 大幅优化了大型 C++ 项目的后端编译性能体验]] `%%textbf4.%%textbf跨模块优化深度`: 全量合并链接优化 (Monolithic LTO) → 摘要驱动轻量链接优化 (ThinLTO) [Monolithic LTO 全局合并所有 Bitcode, 信息绝对完备, 跨模块函数内联及死代码消除最彻底 → 但多模块合并会导致链接期内存爆炸, 耗时巨大, 无法并行: ThinLTO 通过轻量摘要表, 仅局部按需导入关联的 IR 数据, 支持全核心并行编译 → 损失了极其微弱的超全局死代码分析精度 [牺牲了极限状态下的全量分析, 换取了链接时间暴跌与生产环境可用性的飞跃]]

[M3.3] 循环优化 LoopSimplify/LICM 与 ScalarEvolution

循环环境为优化的基石, LLVM 具有一套完备的循环优化 Pass: `ExitBlock` 处理块级退出块 (LoopSimplify); 为循环构建架构单侧的入口前驱块 (Preheader), 唯一的出口块 (Exit Block) 和唯一的返回块 (Latch Block), 消除冗余 CFG; `ExitBlock` 处理循环不分支的侧链 (Latch) 返回到循环体。如果某条指令的操作数在循环体前从未发生变化, 且该值不会副作用, 其在分支在退出块上必确定执行, 则将其提升移动到 Preheader。退出循环的分支重执行。在 `ExitBlock` 处理量级量分析 (Scalar Evolution, SCEV) 用于分析循环中变量变化 (如循环中的索引) 的数学表达式。SCEV 将变量表示为 $\{Start + i, Step\}$ 的多项式形式, 便于判定迭次数、执行次数变化对齐。

[M3.4] 经典冗余消除 EarlyCSE/ADCE 与 GVN 等价判定

冗余计算分配是提升执行效率的最直接手段：`\texttt{tblxfr}\texttt{EarlyErase}`(已在公表子表支持)：基于无配树的前序遍历，维护一个哈希表记录当前已存表的表达式(如`\texttt{x3} = \texttt{add } 132\ \texttt{x1}, \texttt{x2})。当运配子树节点中再次出现该前缀 add 表达式时，直接输出已有的寄存器 \texttt{x3} 值，消除多余运算。\texttt{tblxfr}\texttt{BDFCDE}(侵略性更强代码消解)：假设所有操作符都满足结合律，那么对于任意表达式，其内部必然存在某些子表达式，它们作为指令为后续点，沿 Use-Def 链反向传播。只保留寄存器的数据流值信息，其余无关指令全部删除。\texttt{tblxfr}\texttt{MergeGNN}(全局值编号)：为程序中所有表达式计算一个唯一的全局编号，与 \texttt{EarlyErase} 相比，它支持跨控制流分支判定等价性(如通过图分析，发现不同分支计算出的最终表达式数值等价)，予以合并。`

[M3.5] SimplifyCFG 控制流简化与 Inliner 函数内联

前向控制流图与减少子网调用开销存在非常紧密的耦合。在 SimplicifyCFG 中，`nextblock` 合并只有一个后继块的节点并执行转接。`nextblock` 将只有一个后继的空基块合并为基量。`nextblock` 将只含有一个 phi 节点的分支结构（如 `if-then-else` 且两端都是变量）提升为友方文的 `select` 指令。利用 CPU 的参数传递作优化化分叉预测。`nextblock` 及 `nextBlockInliner`_{inlined} 根据被调函数的尺寸、调用频次（是否仍在循环内以及其它控制流量），计算一个内部代价估值函数 (`InlineCost`)。`nextblock` 若代价值低于阈值，则将调用该函数的 IR 复制并插入在调用处，物理源函数调用指令、参数压栈及返回开销，同时打通跨函数调用的数据流优化。

[M4.1] CodeGen 代码生成流水线阶段划分

LVMH 后端 (CodeGen) 是一个将高层 LIR 变换为具体硬件指令的多阶段物理工厂; `twbfltr` 后端阶段一: `SelectionAndGAGISel` 将 LVMH LIR 树, 翻译为多目标无环的 `SelectionAndGAG` 选择图表示; `twb2b2l` 后端阶段二: `twb2b2l` 类型操作合法化 (Legalization), 将目标硬件不支持的数据类型和推理操作合法化; `twb2b2l` 后端阶段三: `twb2b2l` 指令选择 (Instruction Selection), 通过启发式算法将 `SelectionAndGAG` 选择图转换为目标硬件指令; `twb2b2l` 后端阶段四: `twb2b2l` 调度 (Scheduling), 平铺 `GAG`, 将其转换为硬件 `MachineIR` 序列; `twb2b2l` 后端阶段五: `twb2b2l` 寄存器重分配 (RegAlloc), 将无限的虚拟寄存器映射为物理寄存器; `twb2b2l` 后端阶段六: `twb2b2l` 与 MC 层发射, 构建物理代码并输出汇编二进制文件。

[M4.2] SelectionDAG 构建与类型/操作合法化

SelectionDAG 是一种用于无环图 (DAG) 表达基本块内部数据依赖和控制依赖的物理结构。它由 `SelectionDAG` 结构 (节点 (SDNode) 表示操作, 边 (SDValue) 表示数据流), 此外还包含其他的“链边 (Chain Edge)”以强制约束某些操作指令 (如 `LoadStore`, 其他执行顺序), 则称为 `Chain`。它由 `SelectionDAG` 类型类 (Type Class) 和 `SelectionDAGBuilder` (Promote) 接口组成。如果硬件是 32 位, 则使用 18 条链边为 32 位进行运算。它由 `SelectionDAGSplit` (Split) 接口, 如果硬件是 32 位则 IR 存在 164, 将 164 分解为两个 32 字节计算。它由 `SelectionDAGBuilder` 操作类 (Operation Legalize): 如果目标 CPU 缺少硬件条件, 则在中将 `sdsv` 节点替换为运行时辅助函数调用 (如 `_divdi3`)。

[M4.3] TableGen 匹配表驱动的 ISEL 指令选择

指令选择 (Isel) 将目标相关的 SelectionDag 节点折价为真实机器结构的物理过程。图 1 展示了 `!txt2D32` TableGen 开发者在目标描述文件 (`.td`) 中编写寄存器模式匹配规则 (如定义一个 X86 的 `ADD32rr` 指令, 说明与匹配代码的 `(add i32:$src1, i32:$src2)` 模式)。图 2 展示了 `!txt2D32` 编译期生成: TableGen 工具自动将这些规则编译为一个字节码模式匹配器。图 3 展示了 `!txt2D32` 运行期匹配: Isel 阶段运行一个状态机匹配引擎, 遍历 SelectionDag。当匹配到对应指令模式时, 自动用物理指令节点 (如 `ADD32rr`) 替换原先目标无关的 `SDNode`, 实现指令折叠。

[M4.4] 指令调度 DAG Scheduling 与 MachineInstr 平铺

SelectionDAG 在选择之前需要被铺平为线性的执行序列以供 CPU 执行：`%t12b.t12bf.t12bf.t12bf` 变成了 `SelectionDAG` 是一个有向无环图，而 CPU 的执行流是线性顺序的。`%t12b.t12bf` 指令调度：调度器（如 `ScheduleAGIMLive`）使用启发式策略（基于流水线性延迟、寄存器压力等因素），遍历 DAG 节点，寻找一个能使 CPU 气泡最少、同时寄存器存在生命周期最短的线性序列。`%t12b.t12bf` `MachineInstr` (MI) 调度完成后，DAG 节点解析，转换为线性双向链表组织的通用机器指令 `MachineInstr` 形式，重新存入虚拟寄存器。

[M4.5] GlobalISel 快速流水线通用机器指令 gMIR

为了解决 SelectionDAG 构建及多次 Legalize 寄存器内存占用大、编译速度缓慢的弊端，LLVM 引入到了 GlobalSel（全局指令选择）：`%tctxbl1.%tctxbgMIR(Generic MIR)`；不构建冗余的 SelectionDAG，直接将由 LLVM 翻译成用机器语言 eMIR（如 %vreg0 = G_ADD %x2, %vreg1, %vreg2），在整个 CodeGen 中保持线性 MIR 类选择。`%tctxbl2.%tctxbgMIR(Legalized MIR)`；在 Legalize 和 InstructionSelect 阶段，使用 MachineFunctionPasses。它们可以跨基本块进行全分析并优化，相比 SelectionDAG 仅能在基本块内处理，表现出更强大的全局化能力。`%tctxbl3.%tctxbgMIR` 编译效率率：大幅降低在分配常量，编译吞吐量相较 SelectionDAG 有数倍的提升，尤其适用于 JIT 和 Debug 快速编译场景。

[M5.1] MachineIR (MIR) 结构、虚拟/物理寄存器类

Machine是后端优化的核心容器，它无限逼近该虚拟汇编代码，但在寄存器分配前依然拥有大量虚拟寄存器。如：\textbf{vtxb1f}、\textbf{vtxb1f}结构体次：最外层是MachineFunction，包含MachineBasicBlock(MBB)和MachineInstr(MI)。\textbf{vtxb2f}、\textbf{vtxb3f}MachineOperand：指令内的操作数可以是虚拟寄存器或寄存器（\textbf{Zatack_val}，物理寄存器或寄存器类或内存寻址符）。\textbf{vtxb2f}、\textbf{vtxb3f}RegisterClass：虚拟寄存器必须绑定到一个物理寄存器类（如x86的GR32对应32位通用寄存器。寄存器类定义了该虚拟寄存器在分配时可选用的物理寄存器集合。

[M5.2] RegAllocGreedy 着色干涉图、溢出与 Spill 插入

LYM4 默认采用高度优化的焚焚寄存器分配器 (RegAllocGreedy) 将虚拟寄存器映射到物理寄存器上。VecXtblBt1 活跃区间分析 (Live Intervals): 计算每个虚拟寄存器活跃区间的起始和结束位置, 并据此对活跃寄存器进行排序。VecXtblBt1 活跃区间分析判断两个虚拟寄存器的活跃区间是否重叠。如果重叠, 则在干涉图中建立一条边。判断二者不能共享同一个物理寄存器。VecXtblBt1 着色与 VecXtblBt1 以优先级排序。活跃长度越长、循环内引用次数或变化量越大者越高, 依次为不可用物理寄存器的候选。VecXtblBt1 着色和寄存器分配最终导致函数无表着色。判断是否发生溢出。VecXtblBt1 溢出检测: 在 VecXtblBt1 活跃区间分析的基础上, 对每个活跃寄存器, 在其分配一个插槽 (Stack Slot), 并在该变量的定义点后插入 store 写 (写入内存), 在其使用点前插入 load 从 (从内存载入内存), 并分析该活跃区间是否发生内存溢出。

[M5.3] PEI 栈帧布局重建与 Callee-Saved 恢复

在寄存器分配完成后,所有的虚拟寄存器均被替换为物理寄存器,但帧地址尚未真正生成。`%eax2b`、`%eax2c`、`%eax2d` 帧地址重生成 (PrologEpilogueRegister, PER): `%eax2b` 计算并保存真实地址, `%eax2c` 保存 `%eax2b` 的副本, `%eax2d` 保存 `%eax2b` 的副本。帧地址重生成由 `%eax2b` 根据目标标号的对齐要求 (如 `x86_64` 要求帧指针 16 字节对齐) 确定帧指针偏移量。`%eax2b`、`%eax2c` 的寄存器生成: `%eax2b` 在函数的前 10 个基本块 (Entry) 寄存器, 插入 Prologue 汇编 (如调整 SP 指针 `%rsp` `$32`, `%rsp`, 并保存寄存器 CFP)。 `%eax2b` 在函数的所有返回 `ret` 指令, 插入 Epilogue 汇编 (恢复 SP 并恢复 CFP-Saved 寄存器)。 `%eax2c` 的寄存器生成: `%eax2c` 在函数的前 10 个基本块 (Entry) 寄存器, 插入 Prologue 汇编 (如 `%rbx`, `%r12`)。必须在 Prologue 中将 `%push` 指令, 在 Epilogue 中 `%pop` 指令恢复, 保证调用者环境不被污染。

[M5.4] TableGen .td 物理架构寄存器与指令定义

TableGen是整个LLVM代码生成器在符号物架构树中的顶层扩展。它使用**TextualTableGen**语言：是一种强类型的记录式声明语言，它的语法规则与C++类似，但只支持常量表达式。TableGen的目标CPU有寄存器、寄存器的名称、别名、重要关系（如EAX是RAX的右侧32位）。**Textueltext/textuel**命令描述这些帮助信息（如add），二进制机器码特征、操作数列表（输入输出类型）、执行延迟及隐式寄存器修改写作用。**Textueltext/textuel**调用出如何描述在流水线无阻塞的寄存器（如x86-64的%rdi,%rax,%rdx等），指出如何描述在流水线上阻塞的寄存器（如x86-64的%eax）。

TableGen的编译器使用LLVM的编译器基础设施，它使用LLVM的指令集来读取.c文件，在编译时自动生成主模块的C++模式匹配库和头类型代码，无缝挂接到CodeGen的各个环节。

[M6.1] MC 层 MCStreamer/MCInst 指令流式编码与反汇编

MC (Machine Code) 是 LLVM 工具链中负责最终生成目标文件与可执行文件的终端阶段。`MCInst` 是 LLVM 指令抽象类，它封装了所有编译选项与架构相关的控制流和寄存器信息，仅仅包含寄存器 ID、立即数和寄存器名称，是理解指令的最纯粹抽象。`MCInstStream` 提供了一种统一的流式数据接口。如果输入输出文件，它会将 `MCInstStream` 格式化打印成人可读文本；如果输入输出二进制目标文件，它会将 `MCInstStream` 指令流传递给后端处理器。`MCInst` 流式化生成/反汇编：编码器流式计算指令字节码；反汇编器接收物理二进制数据流并根据 CPU 生成的匹配模式为 `MCInst` 打印，实现双向映射。

[M6.2] 目标二进制 ELF/Mach-O 段生成与符号重定位表

MC 层的 ObjectiveWriter 负责将 MCInst 流封装成操作数能识别的标准指令格式 (Linux 的 ELF, macOS 的 Mach-O、Windows 的 COFF) 作为 `Executable` 生成代码。将代码区 (Text) 的二进制数据封装, 将常量数据放入 `.rodata` 段, 将全局变量写入 `.data` 段, 将局部变量写入 `.bss` 段, 将重定位数据放入 `.rela` 段。在生成可执行文件时, 可执行文件中 `.text` 段中, 每记录每字节对应段内的字节编号为 `Executable` 的 `Relocations` 成员。`Executable` 用于在目标代码编译时, 调用外部函数 (如打印) 或者访问全局变量的物理地址是未知的。`Executable` 内部部含有在调函数处填充占位符, 并在 `.rela` 段重定位数据中生成一条记录, 说明“在模块的哪一个字节修改, 引用了哪一个外部符号”。在链接时必须使用何种寻址模式 (如相对寻址 `R_X86_64_PC32`) 进行地址修正。

[M6.3] OrcJIT 异步编译模型、JITLink/RuntimeDyld 动态链接

LLVM 提供的 JIT (Just-In-Time) 引擎支持在进程运行期间动态将 IR 编译并加载到内存中。使用 `jitrtbl`、`jitrtblOrCjit` 架构; 采用基干 Layer (层) 的模块化堆叠设计。最高层是 LLJIT 引擎接口, 底层是负责加载 IR 的 `IRCompiler`, 再到底层是编译器码并分配内存的 `ObjectLinkingLayer`。LLJIT 引擎支持在 JIT 编译时, `OrCJIT` 允许注册新的 IR 指令, 并支持在 JIT 编译时调用用户自定义的函数指针, 并支持在 JIT 编译时按需编译与加载, 极大地缩短了启动时间。`jitrtbl`、`jitrtblOrCjit` 新一代 JIT 内存链接技术, 取代了旧版的 `RuntimeDyld`。它直接在内存中执行目标文件的解析, 并模拟操作系统的一页一页链接过程; 在物理内存中为代码区申请 `zx` (可执行代码) 页, 为数据区申请 `rw` 页, 然后根据页面地址直接往内存地址填入指令中, 最后完全交付执行。

[M6.4] LTO & ThinLTO 跨模块优化与 Bitcode 局部修剪

LLVM 率先在业界大规模普及并商用了链接时优化 (Link-Time Optimization, LTO) 技术。接下来, LTO 传统编译流程: 传统编译器以模块 (.cpp) 为单位优化, 链接器仅做符号链接, 无法跨模块内联。在链接时, ThinLTO 驱动器直接将模块汇编为二进制, 并输出 LLVM IR Bitcode。在链接时, LTO 驱动器将所有的 bitcode 读入同一内存, 并做全局优化。在生成可执行文件时, 驱动器将所有的 bitcode 读入同一内存, 做极大、模块级 ThinLTO (线程级 LTO): `thinlto.exe` 将 LTO 的第一阶段 (生成单模块 bitcode) 为每个模块生成 bitcode 的同时, 生成一个轻量级的摘要符号表 (包含函数名、全局变量出入参名等)。`thinlto.exe` 的第二阶段: 在链接时, ThinLTO 驱动器快速加载所有模块并做分析。如果发现 A 模块需要内联 B 模块的某函数, 只将 B 模块的该函数 IR 进行局部重入, 而无需合并整个 B 模块。这使得跨模块优化可以在多个 CPU 核心上完全并行执行, 其内联和编译时间开销与通常常规链接, 而性能则等同于 `Monolithic LTO`

Zone T1: LLVM CLI Commands & Debug Flags

clang++ -O3 -mllvm -print-after-all main.cpp：追踪中端所有 Pass 变化。在每个中端 Pass 运行完毕后，在标准输出打印出当前最新的整个 IR 文本，便于在数千个优化 Pass 中精准定位是哪个 Pass 污染了代码或者制造了 Bug。clang++ -Xclang -ast-dump -feyntax-only main.cpp：强制调用 Clang 语法树 (AST)。在终端流式输出彩色缩进的抽象语法树，包含节点的内存地址、作用域范围、类型定义 (如 ParamVarDecl)。用于排查降级生成 IR 前的前端解析正确性。llvm -passes='mem2reg,simplifycfg' -S input.ll -o output.ll：独立调用中端优化器。使用新一代 NPM 执行指定的优化组合 (本例为提升 SSA 再简化控制流)，输出优化后的文本格式 .ll IR 文件。llvm -O2 -mcx86-64 -mcpu=skylake -print-after-isel input.ll：调试后端代码生成。将 LLVM IR 翻译为 X86 汇编，并输出 ISel 模式匹配完成后、机器指令平铺前的底层机器 DAG 节点和通用机器指令序列。

Zone T2: LLVM IR Assembly Patterns & SSA Features

在 LLVM 文本汇编中，可以清晰观测到中端优化的核心指令模式：`%%xtxrf` 变量定义与 `Mem2Reg` 优化对比：\在未开启优化的 IR 中，局部变量（如 C 中的 `int a = 5`）表现为频繁的字：`%a = alloca i32, align 4`；在当前物理地址分配 4 字节 `store i32 %a, ptr %a, align 4`；将立即数 5 写入栈内存处 `%1 = load i32, ptr %a, align 4`；从栈内存处 `%1` 读取到虚拟寄存器 `%1 = add nsw i32 %1, 10`；执行加法运算 `%1` 经过 `Mem2Reg` 优化后，上述代码会被彻底重命名，消除栈空间分配和读写；`alloca`, `store`, `load` 指令被全部抹去 `%add = add nsw i32 5, 10`；常量值直接通过 `alloca` 关系下推，直接生成常量表达式 `%%xtxrfPhi` 节点与控制流汇聚特征：`if` 存在分叉的结构中，不同路径可能对同一个逻辑变量 `%val` 进行了多次 `store`。在合并块 `%merge` 的首部，通过 `phi` 指令将不同的前驱来源收敛：`%then = %val, %val_then = add i32 %x, 1, %lbr label %merge`；`%else = %val, %val_else = sub i32 %x, 1, %lbr label %merge`；`%merge`；`%phi` 语法规：根据前驱基本块的 `actual` 执行路径，挑选对应的寄存器值 `%val = phi i32 [%val_then, %then], [%val_else, %else]` `irret i32 %val`